Assignment-13.3

Ht.No : 2303A51584

Batch: 05

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Outcome

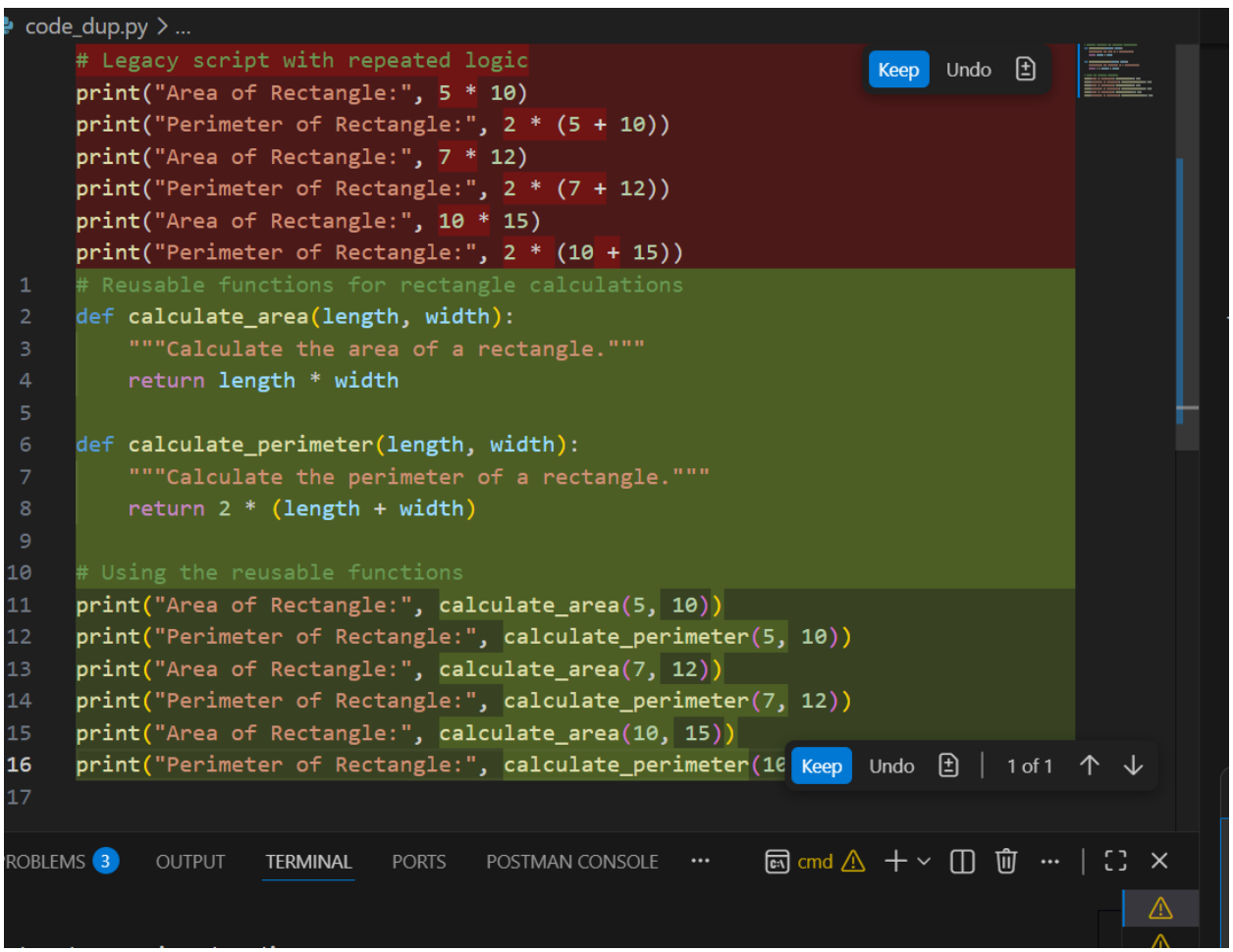
• A reusable function to calculate area and perimeter

- No duplicated code blocks
- Proper docstrings for all functions

Prompt:

Refactor the following Python code by removing repeated logic.
Create reusable functions to calculate the area and perimeter of a rectangle using length and width as parameters, and update the code to use these functions.

Code:



```python
# Reusable functions for rectangle calculations
def calculate_area(length, width):
    """Calculate the area of a rectangle."""
    return length * width

def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle."""
    return 2 * (length + width)

# Using the reusable functions
print("Area of Rectangle:", calculate_area(5, 10))
print("Perimeter of Rectangle:", calculate_perimeter(5, 10))
print("Area of Rectangle:", calculate_area(7, 12))
print("Perimeter of Rectangle:", calculate_perimeter(7, 12))
print("Area of Rectangle:", calculate_area(10, 15))
print("Perimeter of Rectangle:", calculate_perimeter(10, 15))
```

Output:

```
PROBLEMS 3   OUTPUT   TERMINAL   PORTS   POSTMAN CONSOLE

E:\3-2\AI Assisted Coding>C:/Users/hp/AppData/Local/Programs/Python
"e:/3-2/AI Assisted Coding/code_dup.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Explanation:

A function is created to calculate the area and perimeter of a rectangle.

- The function takes length and width as inputs.

• A docstring is added to explain what the function does.

• Inside the function, the area and perimeter are calculated.

• The results are printed.

• The function is called three times with different rectangle values.

• This removes repeated code and makes the program clean and easy to reuse.

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements

using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:

found = False

for n in names:

if s == n:

found = True
```

if found:

print(f"{s} is in the list")

else:

print(f"{s} is not in the list")

Prompt:

Refactor the following Python code to improve performance by removing inefficient nested loops. Use an optimized approach such as set lookups or list comprehensions to check whether each name exists in the list.

Code:

loopRemoval.py > ...

```python
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Optimized approach: Convert to set for O(1) lookup time
names_set = set(names)

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            if found:
                print(f"{s} is in the list")
            else:
                print(f"{s} is not in the list")
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Keep Undo

Keep Undo 2 of 2

```
loopRemoval.py > ...
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Optimized approach: Convert to set for O(1) lookup time
names_set = set(names)

for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Output:

```
oding/loopRemoval.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Explanation:

A list called names stores some names: Alice, Bob, Charlie, and David.

• Another list called search_names contains the names we want to check: Charlie, Eve, and Bob.

• The outer loop goes through each name in search_names.

• A variable found is set to False at the beginning. This means the name is not found yet.

• The inner loop checks each name in the names list.

• If the searched name (s) matches a name in the list (n), found becomes True.

• After checking all names:

• If found is True, the program prints "name is in the list".

• If found is False, the program prints "name is not in the list".

• This process repeats for every name in search_names.

• The code works, but it is slow because it uses nested loops (loop inside another loop).

Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

```
price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

Prompt:

Refactor this Python code by extracting the repeated price and tax calculation into a reusable function calculate_total(price). Make the code cleaner and add proper documentation.

Code:

reusablefun.py > ...

```python
def calculate_total(price):
    """Compute the total amount including an 18% tax.

    Args:
        price (float): The base price of the item.

    Returns:
        float: Total price after adding 18% tax.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    return price + tax

# Examples of using the reusable function
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
print("Total Price:", calculate_total(price))

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
print("Total Price:", calculate_total(price))
```

Keep Undo

Keep Undo

Keep Undo 3 of 3

```
reusablefun.py > ...
def calculate_total(price):
    """Compute the total amount including an 18% tax.

    Args:
        price (float): The base price of the item.

    Returns:
        float: Total price after adding 18% tax.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    return price + tax

# Examples of using the reusable function
price = 250
print("Total Price:", calculate_total(price))

price = 500
print("Total Price:", calculate_total(price))
```

Output:

```
E:\3-2\AI Assisted Coding>C:/Users/hp/AppData/Local/Programs/Python/Python311/p
oding/reusablefun.py"
Total Price: 295.0
Total Price: 590.0
```

Explanation:

The code calculates the total price of a product including tax.

• First, a price value is given (250).

• The tax is calculated as 18% of the price.

• The total price is found by adding the price and tax.

• The program then prints the total price.

• The same steps are repeated again for another price value (500).

• This code has repeated calculations, so it can be improved by creating a function to reuse the calculation.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

• Constants such as PI and RADIUS

• Cleaner, maintainable code

Prompt:

Refactor this Python code by replacing hardcoded values with named constants such as PI and RADIUS to improve readability and maintainability.

Code:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
# Named constants for readability
PI = 3.14159
RADIUS = 7

# Calculations using constants instead of magic numbers
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Keep Undo

```
# Named constants for readability
PI = 3.14159
RADIUS = 7

# Calculations using constants instead of magic numbers
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Output:

```
E:\3-2\AI Assisted Coding>C:/Users/hp/AppData/Local/Programs/Pyth
oding/rplceHardcoded.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation:

The code calculates the area and circumference of a circle.

- The value 3.14159 is used as pi (π).
- The value 7 is used as the radius of the circle.
- The first line calculates the area of the circle.

• The second line calculates the circumference of the circle.

• The program then prints the results.

• The numbers 3.14159 and 7 are hardcoded values.

• These values should be replaced with constants like PI and RADIUS to make the code cleaner and easier to maintain.

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

Expected Outcome

• Descriptive variable names

• Inline comments explaining logic

• Identical output

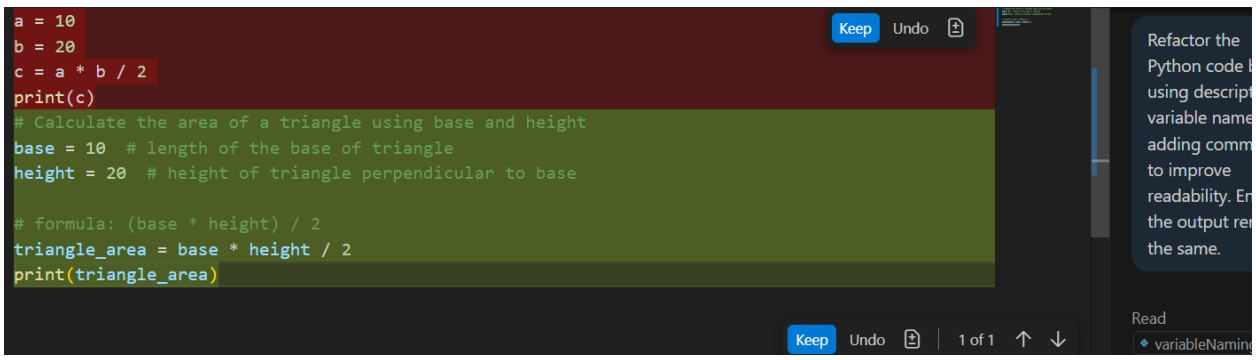
Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if

required, screenshots.

Prompt:

Refactor the following Python code by using descriptive variable names and adding comments to improve readability. Ensure the output remains the same.

Code:




```
# Calculate the area of a triangle using base and height
base = 10  # length of the base of triangle
height = 20  # height of triangle perpendicular to base

# formula: (base * height) / 2
triangle_area = base * height / 2
print(triangle_area)
```

Output:

```
E:\3-2\AI Assisted Coding>C:/Users/hp/AppData
oding/variableNaming.py"
100.0
```

Explanation:

The code assigns 10 to variable a and 20 to variable b.

- It calculates the value of c by multiplying a and b, then dividing by 2.
- The calculation follows the formula:
- area = width * height / 2
- The result is stored in variable c.
- Finally, the program prints the value of c.
- The variable names a, b, and c are not descriptive, so they should be replaced with clear names to improve readability.